Software Engineering 2
Lösung der Übung: REST-Service

Dr. F.-K. Koschnick, Sybit GmbH

Copyright © 2024 Dr. F. K. Koschnick, Sybit GmbH. All rights reserved.

Aufgabe (Story): REST-Service für Retros

Als Schnittstellennutzer möchte ich über einen REST-Service die Retros für ein beliebiges Projekt abgreifen können, um mit den Daten Auswertungen durchführen zu können.

Verifiziere, dass

- mit einem GET-Aufruf der URL /projects/{id}/retros alle Retros zu dem Projekt mit der entsprechenden ID abgerufen werden können

- mit einem GET-Aufruf der URL /retros alle Retros von allen Projekten abgerufen werden können

- mit einem GET-Aufruf der URL /retros/{id} eine Retro mit einer speziellen id abgerufen werden kann

- mit einem POST-Aufruf der URL /retros eine neue Retro für ein spezielles Projekt erzeugt werden kann (gibt zusätzlich 10 Punkte, wer diesen Punkt erledigt, Tipp: Hierbei sollte im Retro-Entity das Attribut id mit 0L vorbelegt sein)

- das Mediaformat der Daten JSON ist

- der Service nicht mit der Session abgesichert ist

<u>Technische Bemerkung:</u>

REST-Controller mit Spring-Boot bauen und den RetroService nutzen.

```
@ManyToOne(fetch = FetchType.EAGER, optional = false)
  @JoinColumn(name = "project_id", nullable = false)
  private Project project;
```

Controller


```
package de.htwg.retroweb.controller.rest;

import java.util.ArrayList;
import java.util.List;

import jakarta.servlet.http.HttpServletResponse;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.MediaType;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;

import de.htwg.retroweb.exception.ResourceNotFoundException;
import de.htwg.retroweb.model.Projects;
import de.htwg.retroweb.model.Retros;
import de.htwg.retroweb.service.ProjectsService;
import de.htwg.retroweb.service.RetrosService;
@RestController
public class ExerciseRetrosRestController {

   @Autowired
private RetroService retroService;

   @Autowired
   private ProjectService projectService;

.......

}
```

Annotation, dass es sich um einen Rest-Controller handelt!!!

Autowired nicht vergessen

Hier kommen die Funktionen mit dem Requestmapping hin

Ergebnis in Postman bei Aufruf: http://localhost:8080/retroweb/projects/2/retros

GET | http://localhost:8080/retroweb/projects/2/retros | Params | Send | Save

Status: 200 OK

Pretty | Raw | Preview | JSON


```
[
    {
        "created": "2018-08-10T13:38:58.000+0000",
        "updated": "2018-08-10T13:38:58.000+0000",
        "id": 2,
        "name": "retro2",
        "active": false,
        "scheduled": "2018-08-17T13:30:00.000+0000",
        "project": {
            "created": "2018-08-10T13:38:58.000+0000",
            "updated": "2018-08-10T13:38:58.000+0000",
            "id": 2,
            "name": "project2",
            "active": true
        }
    },
    {
        "created": "2018-08-10T13:38:58.000+0000",
        "updated": "2018-08-10T13:38:58.000+0000",
        "id": 3,
        "name": "retro3",
        "active": true,
        "scheduled": "2018-09-05T13:00:00.000+0000",
        "project": {
            "created": "2018-08-10T13:38:58.000+0000",
            "updated": "2018-08-10T13:38:58.000+0000",
            "id": 2,
            "name": "project2",
            "active": true
        }
    }
]
```


retro2

retro3

Array von Retros

Response-Code 200

Das Projekt mit der id=2 hat zwei Retros.

Ergebnis in Postman bei Aufruf: http://localhost:8080/retroweb/projects/77/retros

GET http://localhost:8080/retroweb/projects/77/retros Params Send Save

Status: 404 Not Found

Pretty Raw Preview JSON

```
1 []
```

Leerer Array, das Projekt mit der id=77 wurde nicht gefunden, also keine Retros.

Response-Code 404

Problem mit Lazy Loading bei Entity Retro


```
@RequestMapping(value = "/projects/{id}/retros", method = RequestMethod.GET, produces = MediaType.APPLICATION_JSON_VALUE)
  public List<Retro> getRetrosFromProject(@ PathVariable(value = "id"), HttpServletResponse response) {
    response.setHeader("Cache-Control", "no-cache");
    return retroService.getByProjectId(id);
  }
```

org.springframework.http.converter.HttpMessageConversionException: Type definition error: [simple type, class org.hibernate.proxy.pojo.javassist.JavassistLazyInitializer]; nested exception is com.fasterxml.jackson.databind.exc.InvalidDefinitionException: No serializer found for class org.hibernate.proxy.pojo.javassist.JavassistLazyInitializer and no properties discovered to create BeanSerializer (to avoid exception, disable SerializationFeature.FAIL_ON_EMPTY_BEANS) (through reference chain: java.util.ArrayList[0]->de.htwg.retroweb.entities.Retro["project"]->de.htwg.retroweb.entities.Project_$$_jvst5e_0["handler"])

Lazy Loading-Problem:
Wenn die Serialisierung nach JSON beim Return in der Methode startet, ist die Property project noch nicht geladen, es sei denn es ist im Cache von Hibernate, d.h. es wurde vorher schon mal abgefragt.

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "project_id", nullable = false)
private Project project;
```

Lösung 1 für Lazy Loading bei Entity Retro

RestController-Methode

```
@RequestMapping(value = "/projects/{id}/retros", method = RequestMethod.GET, produces = MediaType.APPLICATION_JSON_VALUE)
  public List<Retro> getRetrosFromProject(@ PathVariable(value = "id"), HttpServletResponse response) {
    response.setHeader("Cache-Control", "no-cache");
    return retroService.getByProjectId(id);
  }
```

Lazy-Loading vermeiden bei REST

Retro-Entity

```
@ManyToOne(fetch = FetchType.EAGER, optional = false)
@JoinColumn(name = "project_id", nullable = false)
private Project project;
```

Lösung 2 für Lazy Loading bei Entity Retro


```
@RequestMapping(value = "projects/{id}/retros", method = RequestMethod.GET, produces = MediaType.APPLICATION_JSON_VALUE)
  public List<Retro> getRetrosFromProject(@PathVariable(value = "id") Long projectId, HttpServletResponse response) {
    response.setHeader("Cache-Control", "no-cache");
    Project project;
    try {
      project = projectService.getById(projectId);
      return retroService.findByProject(project);
    } catch (ResourceNotFoundException e) {
      response.setStatus(404);
      return new ArrayList<Retros>();
    }
  }
```

Projekt muss vorher einmal geladen werden, so dass es im Cache von Hibernate ist.

Retro-Entity

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "project_id", nullable = false)
private Project project;
```

RetroService-Methode

```
@Override
  public List<Retro> findByProject(Project project) {
    LOG.debug("findByProject");
    return retroRepository.findByProject(project);
  }
```

getAllRetros


```
@RequestMapping(value = "retros", method = RequestMethod.GET, produces = MediaType.APPLICATION_JSON_VALUE)
  public List<Retro> getAllRetros (HttpServletResponse response) {
    response.setHeader("Cache-Control", "no-cache");
    return retroService.listAll();
}
```

RetroService-Methode

```
@Override
  public List<Retro> listAll() {
    LOG.debug("listAll");
    return retroRepository.findAll();
  }
```

getRetroById


```
@RequestMapping(value = "retros/{id}", method = RequestMethod.GET, produces = MediaType.APPLICATION_JSON_VALUE)
public Retro getRetroById(@PathVariable(value = "id") Long retroId, HttpServletResponse response) {
  try {
    return retroService.getById(retroId);
  } catch (ResourceNotFoundException e) {
    response.setStatus(404);
    return null;
  }
}
```

Alternative, falls Exception mit ResponseCode annotiert

```
@RequestMapping(value = "retros/{id}", method = RequestMethod.GET, produces = MediaType.APPLICATION_JSON_VALUE)
public Retro getRetroById(@PathVariable(value = "id") Long retroId, HttpServletResponse response) throws ResourceNotFoundException {
  return retroService.getById(retroId);
}

@ResponseStatus(value = HttpStatus.NotFound)
public class ResourceNotFoundException extends RetroWebException
```

RetroService-Methode

```
@Override
  public Retro getById(Long id) throws ResourceNotFoundException {
    return retroRepository.findById(id).orElseThrow(() -> new ResourceNotFoundException(RETRO, "id", id));
    //java.util.Optional<T> and Lambda Operator -> without Parameters ()
  }
```

createRetro

Voraussetzung, Exceptions sind mit Response-Code annotiert

```
@PostMapping("retros")
public Retro createRetro(@Valid @RequestBody Retro retro, HttpServletResponse response) throws ResourceAlreadyExistsException, ResourceNotFoundException {
  projectService.getById(retro.getProject().getId());//check if project exists
  response.setStatus(201);
  Retro newRetro = retroService.save(retro);
  response.setHeader("Location", "/retros/" + newRetro.getId())
  return newRetro;
}
```

RetroService-Methode

```
@Override
public Retro save(Retro domainObject) throws ResourceAlreadyExistsException {
  Retro newRetro;
  LOG.debug("--> save");
  List<Retro> retros = retroRepository.findByName(domainObject.getName());
  if(retros.isEmpty()) {
    domainObject.setCreated(new Date());
    newRetro = retroRepository.save(domainObject);
  } else {
    LOG.debug("<-- save, ResourceAlreadyExistsException");
    throw new ResourceAlreadyExistsException(RETRO, "name", domainObject.getName());
  }
  LOG.debug("<-- save");
  return newRetro;
}
```

Retro-Entity

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id = 0L;
```

Bemerkung zu Entities Project und Retro

Um eine Unendlich-Schleife bei der JSON-Generierung zu vermeiden, gibt es folgende Möglichkeiten:

1. Bei *@OneToMany* die Annotation *@JsonManagedReference* verwenden (in Project-Entity bei Property Retros) und bei *@ManyToOne* die Annotation *@JsonBackReference* verwenden (in Retro-Entity bei Property Project). In diesem Fall wird beim Json des Retro-Entity das Project nicht mitgeliefert
2. Im Entity Project einfach Property Retros inklusive Getter und Setter löschen. Dann kann man zwar beim Project-Entity nicht auf die Retros zugreifen, aber beim Json der Retro wird das Project mitgeliefert.